

دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

پروژه پایانی درس سیستم‌های عامل

عنوان:

شبیه‌سازی عملکرد سیاست‌های متفاوت حافظه نهان برای درخواست‌های ورودی خروجی سامانه ذخیره‌سازی

نگارش

امیرکسری احمدی

روژین تقی‌زادگان

شمیم رحیمی

رادین شاه دائی

استاد

دکتر حسین اسدی

بهمن ۱۴۰۳

فهرست مطالب

۱ نحوه استخراج دنباله دسترسی از دیتاست ۲

۲ سیاست جایگزینی LRU ۳

۳ سیاست جایگزینی N-Hit ۵

۴ سیاست جایگزینی NHit-LRU ۱۰

۵ سیاست جایگزینی Oracle/Belady ۱۲

۶ سیاست جایگزینی ARC ۱۵

۷ سیاست جایگزینی LARC ۲۰

۸ نتایج اجرای سیاست‌های مختلف روی حافظه‌ی نهان ۲۲

۹ تحلیل نرخ برخورد ۲۴

۱ نحوه استخراج دنباله دسترسی از دیتاست

پیش از اعمال الگوریتم‌های جایگزینی، نیاز است که در قدم اول، دنباله دسترسی‌ها را از دیتاست‌های واقعی مربوط به علی‌بابا استخراج کنیم. همانظور که در صورت پروژه نیز آورده شده بود، نیاز است از استخراج دنباله‌ها درون یک بازه یا Interval زمانی نیز پشتیبانی کنیم. بدین منظور، تابع get_sequence در ابتدای برنامه نوشته شده است.

```python
def get_sequence(filename, t1, t2):
    min_offset = np.inf
    max_offset = -np.inf
    sequence = []
    with open(filename, "r") as file:
        reader = csv.reader(file)
        for row in reader:
            timestamp = int(row[0])
            if t1 <= timestamp <= t2:
                offset = int(row[2])
                operation = row[4].strip()
                sequence.append((offset, operation))

                if offset < min_offset:
                    min_offset = offset
                if offset > max_offset:
                    max_offset = offset

    return sequence, int((max_offset - min_offset) / (4096 * 100))
```

شکل ۱: تابع مربوط به استخراج دنباله

این تابع، مسئولیت دو چیز را بر عهده دارد. مسئولیت اول استخراج دنباله می‌باشد. همانطور که از نام‌های ستون‌های دیتاست مشخص است، ستون اول مربوط به Timestamp، ستون سوم مربوط به Offset و ستون پنجم مربوط به نوع عملیات یا Operation می‌باشد. همچنین حائز اهمیت است که این دسترسی‌ها، در واقع مربوط به دسترسی‌ها به صحفه‌ها (Pages) هستند و هر دسترسی مربوط به ۴۰۹۶ بایت یا 4KB است. مسئولیت دوم این تابع، مشخص کردن اندازه حافظه نهان است. اندازه حافظه نهان برابر یک درصد از اندازه دامنه آدرس‌ها انتخاب شده است. به منظور پیدا کردن این مقدار، کافی‌ست کمینه و بیشینه Offsetها را پیدا کنیم و در ادامه با تقسیم تفاضل این دو مقدار بر اندازه بلوک‌ها که 4KB باشد، اندازه دامنه آدرس‌ها را پیدا کنیم. توجه کنید که این تابع، از استخراج دنباله مربوط به یک بازه زمانی خاص که با t_1 و t_2 مشخص می‌شود پشتیبانی می‌کند.

۲ سیاست جایگزینی LRU

الگوریتم LRU مشهورترین الگوریتم جایگزینی حافظه‌ی نهان است که پیاده‌سازی آن ساده است و همچنین عملکرد بالایی دارد.

الگوریتم این سیاست مبتنی بر زمانی است که عدم برخورد در حافظه‌ی نهان رخ می‌دهد و حافظه‌ی نهان پر است. در این سیاست برای جایگزینی داده‌ی جدید سراغ داده‌ای می‌رویم که از زمان مورد استفاده قرار گرفتنش بیش از بقیه گذشته باشد.

برای پیاده‌سازی این الگوریتم از یک داده ساختار OrderedDict پایتون استفاده می‌کنیم که در آن هر وقت به یک داده دسترسی جدید انجام شود بتوانیم آن را به آخر لیست ببریم.

نحوه‌ی عملکرد الگوریتم LRU:

۱. برخورد در حافظه نهان (شکل ۲)

- در این بخش ما با بررسی اینکه آفست صفحه‌ی مورد نظر در حافظه‌ی نهان قرار دارد یا نه تعداد برخوردها را یکی زیاد می‌کنیم.

```
if offset in cache:
    # Cache hit
    cache.move_to_end(offset)  # Mark as MRU
    if operation == 'Read':
        total_read_hits += 1
    elif operation == 'Write':
        total_write_hits += 1
```

شکل ۲: کد مربوط به بخش برخورد در حافظه‌ی نهان

۲. عدم برخورد در حافظه نهان (شکل ۳)

- در تصویر مربوط به این بخش نیز می‌توانید مشاهده کنید که ابتدا با توجه به نوع عملیات تعداد خطاهای آن را زیاد می‌کنیم.
- در صورت پر بودن حافظه‌ی نهان آفست داده‌ای که دیرترین دسترسی را داشته از حافظه‌ی نهان پاک می‌کنیم و آفست داده‌ی جدید را اضافه می‌کنیم.

```
else:
    # Cache miss
    if operation == 'Read':
        total_read_misses += 1
    elif operation == 'Write':
        total_write_misses += 1

    # Evict LRU if cache is full
    if len(cache) >= cache_size:
        cache.popitem(last=False)

    # Add to cache
    cache[offset] = None
```

شکل ۳: کد مربوط به بخش عدم برخورد در حافظه نهان

۳ سیاست جایگزینی N-Hit

الگوریتم N-Hit یک سیاست جایگزینی حافظه‌ی نهان است که در آن بلاک‌های حافظه برای ورود به حافظه‌ی نهان باید حداقل یک تعداد مشخصی (insertion threshold) دسترسی داشته باشند.

این موردی که در پاراگراف بالا گفته شد سیاست جایگزینی در حافظه‌ی نهان نیست بلکه سیاست ورود به آن است. برای جایگزینی در این الگوریتم از سیاست‌های مختلفی مانند LRU استفاده می‌شود اما در صورت پروژه از ما خواسته شده که سیاست جایگزینی مبتنی بر کمترین تعداد دسترسی در میان بلوک‌های حافظه‌ی نهان باشد.

نحوه‌ی کار این الگوریتم بدین شکل است که با استفاده از یک لیست تعداد دسترسی‌ها به هر آفست را ذخیره می‌کنیم. یک دیکشنری برای آفست‌هایی که درون حافظه‌ی نهان و تعداد دسترسی‌ها به آن آفست‌ها است نیز وجود دارد. همچنین یک SortedDict به نام sorted_cache داریم که در آن به ازای هر تعداد دسترسی یک لیست از آفست‌ها داریم.

در sorted_cache ما به ازای تعداد دسترسی‌های مختلف آفست‌هایی که در حافظه‌ی نهان قرار دارند یک لیست از آفست‌ها نگه می‌داریم در نتیجه هر بار که بخواهیم عنصری را خارج کنیم به راحتی می‌توانیم آفستی که کمترین دسترسی را داشته (همچنین در صورت تساوی دیرترین دسترسی) را داشته حذف کنیم.

همچنین این الگوریتم یک پارامتر دیگر به نام trigger threshold نیز دارد که در آن مشخص می‌شود تا یک درصد خاص از حافظه‌ی نهان تمامی بلوک‌ها وارد حافظه‌ی نهان شوند و از وقتی که از آن مرز رد شد شروع به اعمال این سیاست کند.

نحوه‌ی عملکرد الگوریتم N-Hit:

۱. پیدا کردن تعداد دسترسی‌های جدید و قدیمی آفست (شکل ۴)

- در این بخش با دسترسی به دیکشنری access_count تعداد دسترسی‌های قبلی را در prev_nhit ریخته و آن را به علاوه یک کردیم و در new_nhit ریختیم.

```
prev_nhit = access_count[offset]

access_count[offset] += 1
new_nhit = access_count[offset]
```

شکل ۴: کد مربوط به بخش دستیابی به تعداد دسترسی‌های آفست

۲. برخورد در حافظه نهان (شکل ۵)

- در این بخش ما با بررسی اینکه آفست صفحه‌ی مورد نظر در حافظه‌ی نهان قرار دارد یا نه تعداد برخوردها را یکی زیاد می‌کنیم.
- همچنین در صورت وجود این آفست در حافظه‌ی نهان باید از لیست sorted_cache لیست مربوط به prev_nhit را به دست آورده و آفست مورد نظر را از آن حذف کنیم.
- همچنین در نهایت به لیست next_nhit درون sorted_cache آفست مورد نظر را اضافه می‌کنیم.

```
if offset in cache:
    cache[offset] = new_nhit

    sorted_cache[prev_nhit].remove(offset)
    if not sorted_cache[prev_nhit]:
        del sorted_cache[prev_nhit]

    if new_nhit not in sorted_cache:
        sorted_cache[new_nhit] = deque()
    sorted_cache[new_nhit].append(offset)

    if operation.lower() == 'read':
        total_read_hits += 1
    elif operation.lower() == 'write':
        total_write_hits += 1
```

شکل ۵: کد مربوط به بخش برخورد در حافظه‌ی نهان

۳. عدم برخورد در حافظه نهان (شکل ۷)

- در تصویر مربوط به این بخش نیز می‌توانید مشاهده کنید که ابتدا با توجه به نوع عملیات تعداد خطاهای آن را زیاد می‌کنیم.
- نسبت مقدار پر شده از حافظه‌ی نهان به کل اندازه‌ی حافظه را با پارامتر trigger_threshold مقایسه می‌کنیم.

- در صورت رد کردن trigger_threshold بررسی می‌کنیم که دسترسی‌های داده از insertion_threshold بیشتر شده یا نه. می‌دانیم در صورتی که از این مقدار کمتر باشد داده نمی‌تواند وارد حافظه‌ی نهان شود.
- در صورت پر بودن حافظه‌ی نهان نیز آفستی که کمترین دسترسی را داشته باشد پیدا می‌کنیم و در صورتی که چند آفست تعداد دسترسی مساوی داشته باشند از الگوریتم LRU بهره می‌بریم. حال آفست پیدا شده را از حافظه‌ی نهان پاک می‌کنیم و آفست جدید را درون آن قرار می‌دهیم.
- همچنین در صورتی که به trigger_threshold نرسیده باشیم داده را بدون اقدام خاصی وارد حافظه‌ی نهان می‌کنیم.

```
else:
    if operation.lower() == 'read':
        total_read_misses += 1
    elif operation.lower() == 'write':
        total_write_misses += 1


    if len(cache) / cache_size >= trigger_threshold:
        if prev_nhit >= insertion_threshold:
            if len(cache) >= cache_size:
                min_nhit, blocks = sorted_cache.peekitem(0)
                evict_offset = blocks.popleft()
                if not blocks:
                    del sorted_cache[min_nhit]

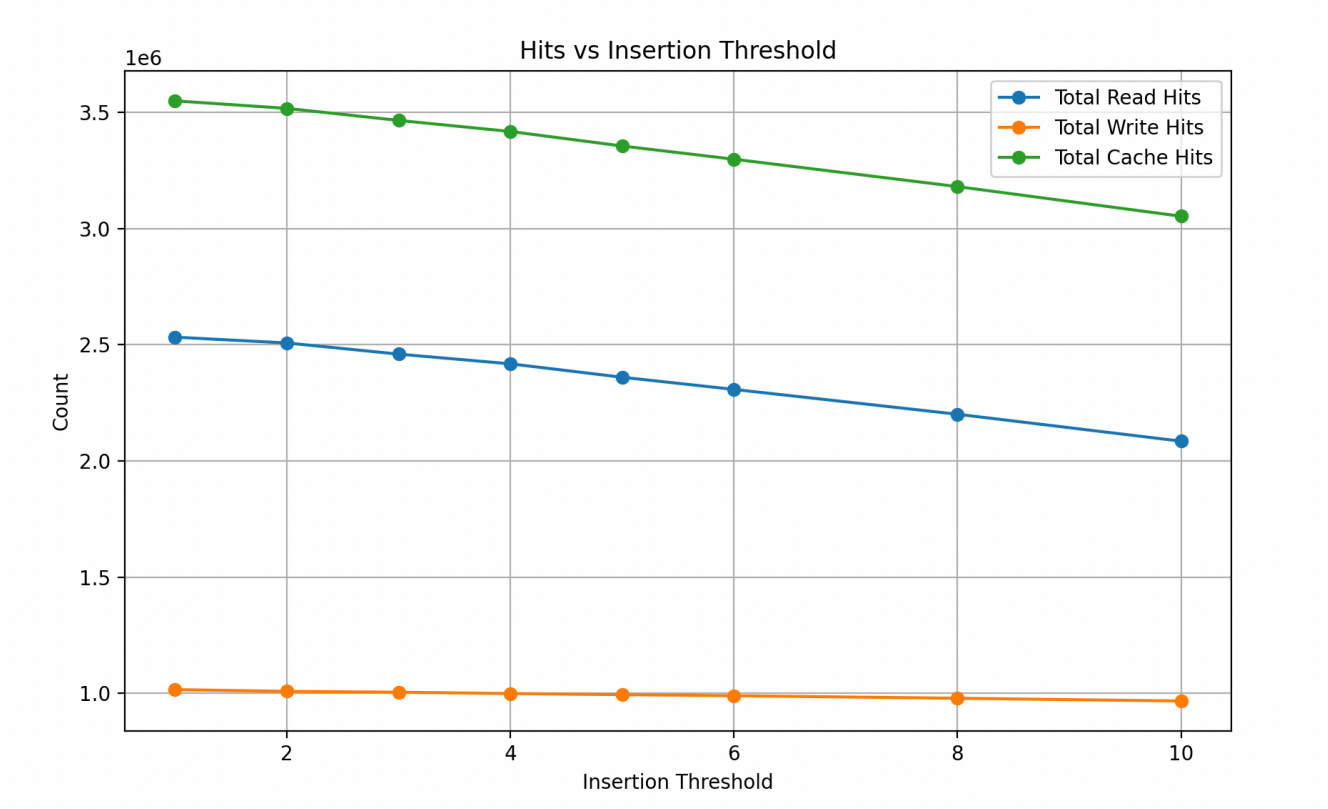
                del cache[evict_offset]

            cache[offset] = new_nhit
            if new_nhit not in sorted_cache:
                sorted_cache[new_nhit] = deque()
            sorted_cache[new_nhit].append(offset)
    else:
        cache[offset] = new_nhit
        if new_nhit not in sorted_cache:
            sorted_cache[new_nhit] = deque()
        sorted_cache[new_nhit].append(offset)
```

شکل ۶: کد مربوط به بخش عدم برخورد در حافظه نهان

تحلیل رفتار سیاست N-Hit روی داده‌های A42: برای تحلیل این الگوریتم، آن را روی داده‌های

موجود با insertion treshold های بین ۱ تا ۱۰ تست می‌کنیم. نتایج زیر مشاهده می‌شوند:



شکل ۷: نتایج شبیه‌سازی N-Hit با Insertion Treshold های مختلف

نتایج نشان می‌دهند که این اجرای این الگوریتم برای داده‌های موجود باعث افزایش نرخ برخورد نمی‌شود. می‌توان نتیجه گرفت که داده‌های مورد آزمایش از دسته‌هایی هستند که این الگوریتم برای آن‌ها به طور بهینه کار نمی‌کند. در ادامه چند مورد از این دسته‌ها را مثال می‌زنیم که داده‌های ما می‌تواند جز یکی از این دسته‌ها باشد:

۱. Non-Stationary Workloads: الگوریتم N-HIT برای تصمیم‌گیری در مورد حافظه‌ی نهان، به الگوهای دسترسی تاریخی متکی است. اگر بارکاری غیرایستا باشد (یعنی الگوهای دسترسی به طور مکرر تغییر کنند)، داده‌های تاریخی ممکن است نتوانند دسترسی‌های آینده را به درستی پیش‌بینی کنند.

تاثیر: الگوریتم ممکن است تصمیم‌های ضعیفی در مورد کش بگیرد که منجر به نرخ برخورد پایین‌تر و کاهش عملکرد کلی شود.

۲. Random Access Patterns: در بارکاری‌هایی که الگوهای دسترسی تصادفی یا غیرقابل پیش‌بینی دارند، N-HIT ممکن است نتواند روندها یا الگوهای معناداری در داده‌ها شناسایی کند.

تاثیر: الگوریتم ممکن است مزیت قابل توجهی نسبت به استراتژی‌های ساده‌تر مانند LRU یا FIFO ارائه ندهد.

۳. Dynamic Workloads with Rapid Changes: در محیط‌هایی که بارکاری به سرعت و به طور غیرقابل پیش‌بینی تغییر می‌کند، N-HIT ممکن است نتواند به سرعت با الگوهای جدید دسترسی سازگار شود.

تاثیر: الگوریتم ممکن است همچنان بر اساس داده‌های تاریخی قدیمی تصمیم‌گیری کند که منجر به عملکرد ضعیف کش می‌شود.

۴ سیاست جایگزینی NHit-LRU

در این قسمت به پیاده‌سازی الگوریتم N-Hit با استفاده از سیستم جایگزینی LRU می‌پردازیم.

به طور کلی در پیاده‌سازی سیاست‌ها برای حافظه‌های نهان یک سیاست برای promotion و یک سیاست برای eviction در نظر گرفته می‌شود. در اینجا پیاده‌سازی سیاست promotion ما همان N-Hit باقی می‌ماند اما سیاست جایگزینی به LRU تغییر می‌کند.

دو پارامتر insertion threshold و trigger threshold نیز مانند بخش قبلی تعریف می‌شوند.

نحوه‌ی عملکرد الگوریتم NHit-LRU:

۱. برخورد در حافظه نهان (شکل ۸)

- در این بخش ما با بررسی اینکه آفست صفحه‌ی مورد نظر در حافظه‌ی نهان قرار دارد یا نه تعداد برخوردها را یکی زیاد می‌کنیم.

```
if offset in cache:
    # Cache hit
    cache.move_to_end(offset)  # Mark as MRU
    if operation == 'read':
        total_read_hits += 1
    elif operation == 'write':
        total_write_hits += 1
```

شکل ۸: کد مربوط به بخش برخورد در حافظه‌ی نهان

۲. عدم برخورد در حافظه نهان (شکل ۹)

- در تصویر مربوط به این بخش نیز می‌توانید مشاهده کنید که ابتدا با توجه به نوع عملیات تعداد خطاهای آن را زیاد می‌کنیم.
- در ابتدا بررسی می‌کنیم که نسبت فضای خالی حافظه به اندازه‌ی آن از مرز مشخص شده عبور کرده یا خیر.
- در صورت عدم عبور از trigger_threshold داده را وارد حافظه‌ی نهان می‌کنیم.
- در صورت عبور از trigger_threshold باید شرط تعداد دسترسی آن را بررسی کنیم. در صورتی که تعداد دسترسی‌های آن از مرز مشخص شده عبور نکرده باشد نمی‌تواند وارد حافظه‌ی نهان شود.

- در صورت عبور تعداد دسترسی‌ها از insertion_threshold باید یک بلاک را از حافظه خارج کنیم که برای آن از الگوریتم LRU استفاده می‌کنیم. در نتیجه بلاک داده با دیرترین دسترسی را از حافظه‌ی نهان خارج می‌کنیم و بلاک جدید را درون آن قرار می‌دهیم.

```
else:
    # Cache miss
    if operation == 'read':
        total_read_misses += 1
    elif operation == 'write':
        total_write_misses += 1

    if len(cache) / cache_size >= trigger_threshold:
        # Only cache the block if it has been accessed at least N times
        if access_count[offset] >= insertion_threshold:
            # Evict LRU if cache is full
            if len(cache) >= cache_size:
                cache.popitem(last=False)

            # Add to cache
            cache[offset] = None
    else:
        cache[offset] = None
```

شکل ۹: کد مربوط به بخش عدم برخورد در حافظه نهان

۵ سیاست جایگزینی Oracle/Belady

الگوریتم جایگزینی Oracle/Belady همان الگوریتم Optimal است. این الگوریتم یک دنباله دسترسی به حافظه را پردازش می‌کند و با توجه به دسترسی‌های آینده در دنباله، بهینه‌ترین تصمیم را برای تخلیه از حافظه نهان می‌گیرد. واضح است که در اندازه حافظه نهان ثابت، این الگوریتم بهترین عملکرد را در میان سایر الگوریتم‌ها خواهد داشت.

گام‌های این الگوریتم به صورت زیر هستند:

۱. مقداردهی اولیه و پردازش درخواست‌های آینده (شکل ۱۰)

ر این مرحله علاوه بر مقداردهی اولیه به متغیرها، هر صفحه به همراه اندیس دسترسی‌های آینده‌اش در دنباله دسترسی در یک دیکشنری ذخیره می‌شود.

```
# Initialize cache as a set for lookups
cache = set()

# Statistics
total_read_hit = 0
total_read_miss = 0
total_write_hit = 0
total_write_miss = 0
total_write_requests = 0
total_read_requests = 0
total_requests = 0
total_cache_hit = 0
total_cache_miss = 0

# Preprocess: Map each page to a list of its future indices
future_uses = defaultdict(list)
for idx, (page, operation) in enumerate(sequence):
    future_uses[page].append(idx)
```

شکل ۱۰: کد مربوط به بخش مقداردهی اولیه

۲. بررسی هر درخواست دسترسی به حافظه

در این حالت دو حالت ممکن است رخ دهد:

- وقوع برخورد در حافظه نهان

در صورت وقوع برخورد در حافظه نهان، نیازی به تغییر در حافظه نهان نیست. (شکل

(۱۱

```
if current_page in cache:
    # Cache Hit
    total_cache_hit += 1
    if operation == "Read":
        total_read_hit += 1
    elif operation == "Write":
        total_write_hit += 1
    # No need to update cache as the page is already present
```

شکل ۱۱: کد مربوط به بخش وقوع برخورد در حافظه نهان

- عدم برخورد در حافظه نهان

در این حالت، در صورتی که حافظه نهان هنوز پر نباشد، صفحه مورد نظر در یکی از خانه‌های حافظه نهان قرار می‌گیرد.

اما اگر حافظه نهان پر باشد، لازم است ابتدا یکی از صفحات موجود در حافظه نهان خارج شود و سپس صفحه جدید در حافظه نهان قرار گیرد. به این منظور، باید از میان تمام صفحات موجود در حافظه نهان، صفحه‌ای که دورترین دسترسی به آن وجود دارد را انتخاب کرده و از حافظه نهان خارج کنیم. (شکل ۱۲)

```
else:
    # Cache Miss
    total_cache_miss += 1
    if operation == "Read":
        total_read_miss += 1
    elif operation == "Write":
        total_write_miss += 1

    if len(cache) < cache_size:
        # If there's space, simply add the page
        cache.add(current_page)
    else:
        # Need to decide which page to evict
        # For each page in cache, find the next use
        next_use = {}
        for page in cache:
            if future_uses[page]:
                next_use_page = future_uses[page][0]
                next_use[page] = next_use_page
            else:
                # If the page is not used again, set its next use to infinity
                next_use[page] = float('inf')

        # Select the page with the farthest next use
        page_to_evict = max(next_use, key=next_use.get)

        # Evict the selected page
        cache.remove(page_to_evict)

        # Add the current page to cache
        cache.add(current_page)
```

شکل ۱۲: کد مربوط به بخش عدم برخورد در حافظه نهان

۶ سیاست جایگزینی ARC

الگوریتم ARC یک سیاست جایگزینی حافظه نهان است که به صورت پویا بین داده‌هایی که اخیرا استفاده شده‌اند و داده‌هایی که بیشترین استفاده را داشته‌اند، تعادل ایجاد می‌کند.

این سیاست از دو لیست اصلی T1 و T2 استفاده می‌کند که به ترتیب مربوط به داده‌هایی که اخیرا استفاده شده‌اند و داده‌هایی که بیشترین استفاده را داشته‌اند، هستند. همچنین از دو لیست جانبی دیگر برای نگهداری صفحاتی که اخیرا از حافظه نهان حذف شده‌اند استفاده می‌شود. لیست B1 شامل صفحاتی‌است که قبلا در T1 بوده‌اند اما از حافظه نهان حذف شده‌اند و لیست B2 شامل صفحاتی است که قبلا در T2 بوده‌اند اما از حافظه نهان حذف شده‌اند.

این دو لیست جانبی به الگوریتم ARC کمک می‌کنند تا بتواند نوع داده‌ای که مورد دسترسی واقع شده (بر اساس تازگی دسترسی یا فرکانس دسترسی) را پس از خروج از حافظه نهان تشخیص دهد و از آن برای تغییر اندازه لیست‌های T1 و T2 استفاده کنند.

تطبیق پویا در الگوریتم جایگزینی ARC به صورت زیر است:

- در صورتی که داده مورد دسترسی پس از خروج از لیست T1 در لیست جانبی B1 باشد، الگوریتم ARC اندازه لیست T1 را افزایش می‌دهد تا به داده‌هایی که اخیرا دیده شده‌اند اولویت بیشتری دهد.
- در صورتی که داده مورد دسترسی پس از خروج از لیست T2 در لیست جانبی B2 باشد، الگوریتم ARC اندازه لیست T2 را افزایش می‌دهد تا به داده‌هایی که بیشترین استفاده را داشته‌اند اولویت بیشتری دهد.

به طور کلی شیوه کار الگوریتم جایگزینی ARC به صورت زیر است:

۱. برخورد در حافظه نهان

- اگر داده مورد دسترسی در لیست T1 باشد، از این لیست خارج می‌شود و وارد لیست T2 می‌شود تا نشان دهد این داده هم‌اکنون به صورت مکرر مورد دسترسی واقع شده است. (شکل ۱۳)

```
if page in T1:
    T1.pop(page)
    T2[page] = True  # Move to T2 as it's now frequently used
    hit = True
    if operation == 'Read':
        total_read_hits += 1
    else:
        total_write_hits += 1
```

شکل ۱۳: کد مربوط به بخش برخورد در لیست T1

- اگر داده مورد دسترسی در لیست T2 باشد، در همان لیست باقی می‌ماند. (شکل ۱۴)

```
elif page in T2:
    T2.move_to_end(page)  # Update position to mark as recently used
    hit = True
    if operation == 'Read':
        total_read_hits += 1
    else:
        total_write_hits += 1
```

شکل ۱۴: کد مربوط به بخش برخورد در لیست T2

۲. برخورد در لیست جانبی

- در صورتی که داده مورد دسترسی در لیست جانبی B1 باشد، الگوریتم ARC متوجه می‌شود که تازگی دسترسی مهم‌تر از فرکانس دسترسی است و اندازه لیست T1 را افزایش می‌دهد. (شکل ۱۵)

```
elif page in B1:
    # Adjust target size p
    p = min(cache_size, p + max(1, len(B2) // max(1, len(B1))))
    B1.pop(page)
    T2[page] = True  # Move to T2 as it's now frequently used
    # Evict if necessary
    if len(T1) + len(T2) > cache_size:
        if len(T1) > p:
            old, _ = T1.popitem(last=False)
            B1[old] = True
        else:
            old, _ = T2.popitem(last=False)
            B2[old] = True
    hit = False
    if operation == 'Read':
        total_read_misses += 1
    else:
        total_write_misses += 1
```

شکل ۱۵: کد مربوط به بخش برخورد در لیست B1

- در صورتی که داده مورد دسترسی در لیست جانبی B2 باشد، الگوریتم ARC متوجه می‌شود که فرکانس دسترسی مهم‌تر از تازگی دسترسی است و اندازه لیست T2 را افزایش می‌دهد. (شکل ۱۶)

```
elif page in B2:
    # Adjust target size p
    p = max(0, p - max(1, len(B1) // max(1, len(B2))))
    B2.pop(page)
    T2[page] = True  # Move to T2 as it's now frequently used
    # Evict if necessary
    if len(T1) + len(T2) > cache_size:
        if len(T1) > p:
            old, _ = T1.popitem(last=False)
            B1[old] = True
        else:
            old, _ = T2.popitem(last=False)
            B2[old] = True
    hit = False
    if operation == 'Read':
        total_read_misses += 1
    else:
        total_write_misses += 1
```

شکل ۱۶: کد مربوط به بخش برخورد در لیست B2

۳. عدم برخورد در حافظه نهان (شکل ۱۷)

- اگر داده جدیدی مورد دسترسی واقع شود و در حافظه نهان نباشد، به لیست T1 افزوده می‌شود.
- در صورتی که اندازه لیست T1 از مقدار تعیین‌شده برای آن بیشتر شود، داده‌ای که دورترین دسترسی به آن انجام شده از لیست T1 خارج می‌شود و وارد لیست جانبی B1 می‌شود.

```
# Cache Miss: Page is not in any list
if len(T1) + len(T2) == cache_size:
    if len(T1) < cache_size:
        if B2:
            B2.popitem(last=False)
            T2.popitem(last=False)
    else:
        old, _ = T1.popitem(last=False)
        B1[old] = True
elif len(T1) + len(T2) < cache_size and (len(T1) + len(T2) + len(B1) + len(B2)) >= 2 * cache_size:
    if len(T1) + len(T2) + len(B1) + len(B2) > 2 * cache_size:
        if B1:
            B1.popitem(last=False)
        else:
            B2.popitem(last=False)

T1[page] = True  # Add to T1 as it's recently used
hit = False
if operation == 'Read':
    total_read_misses += 1
else:
    total_write_misses += 1
```

شکل ۱۷: کد مربوط به بخش عدم برخورد در حافظه نهان

۴. تخلیه

- در صورتی که حافظه نهان پر باشه، الگوریتم ARC داده‌ها را بر اساس تعادل ایجاد شده بین تازگی و فرکانس دسترسی به داده‌ها تخلیه می‌کند.

کد مربوط به بخش تضمین اندازه درست حافظه نهان در شکل ۱۸ آمده است.

```
# Ensure total size constraints
while len(T1) + len(T2) > cache_size:
    if len(T1) > p:
        if T1:
            old, _ = T1.popitem(last=False)
            B1[old] = True
    else:
        if T2:
            old, _ = T2.popitem(last=False)
            B2[old] = True

while len(T1) + len(T2) + len(B1) + len(B2) > 2 * cache_size:
    if len(B1) > p:
        if B1:
            B1.popitem(last=False)
    else:
        if B2:
            B2.popitem(last=False)
```

شکل ۱۸: کد مربوط به بخش تضمین اندازه حافظه نهان

۷ سیاست جایگزینی LARC

LARC یا Lazy Adaptive Replacement Cache، یک الگوریتم جایگزینی مشابه با سیاست جایگزینی ARC است که در بخش قبل توضیح داده شده است. تفاوت این سیاست با سیاست ARC این است که برخی از فرایندهای به‌روزرسانی را به تاخیر می‌اندازد تا سربار پردازشی کمتری داشته باشد و کارایی بهتری در سیستم‌های با بار بالا داشته باشد. نام Lazy این الگوریتم نیز از همین جا می‌آید.

کد و نحوه پیاده‌سازی این الگوریتم، بسیار مشابه الگوریتم ARC می‌باشد. لذا تنها تفاوت‌های سیاست‌های ARC و LARC در این بخش آورده شده است.

۱. تفاوت اول با تاخیر در پاک‌سازی یا Deferred Cleanup یاد می‌شود. در ARC همواره اندازه لیست‌های T1 و T2 و همچنین لیست‌های B1 و B2 کنترل می‌شود تا از مجموع اندازه حافظه بیشتر نشود. اما در LARC این کنترل با تاخیر انجام می‌شود. کد مروبطه به الگوریتم LARC در مورد تاخیر در پاکسازی در را می‌توانید در شکل ۱۹ مشاهده کنید.

```
# Adjust cache if necessary (Deferred Cleanup)
while len(T1) + len(T2) > cache_size:
    if len(T1) > p:
        evict_page, _ = T1.popitem(last=False)
        B1[evict_page] = True  # Add to B1
    else:
        evict_page, _ = T2.popitem(last=False)
        B2[evict_page] = True  # Add to B2

while len(B1) + len(B2) > cache_size:
    if B1:
        B1.popitem(last=False)
    else:
        B2.popitem(last=False)
```

شکل ۱۹: کد مربوط به بخش تاخیر در پاکسازی

۲. تفاوت دوم مربوط به تاخیر در حذف در هنگام Cache Miss می‌باشد. در ARC، وقتی اندازه لیست‌های T1 و T2 برابر اندازه حافظه نهان می‌شود، بالافاصله فضای خالی ایجاد می‌شود. اما در LARC، تا زمانی که ضرورت نداشته باشد، این فضای خالی ایجاد نخواهد شد. کد مربوط به تاخیر در حذف در الگوریتم LARC در شکل ۲۰ قابل مشاهده است.

```
# Cache Miss: Page is not in any list
if len(T1) + len(T2) == cache_size:
    if len(T1) >= 1 and (len(T1) > p):
        evict_page, _ = T1.popitem(last=False)
        B1[evict_page] = True  # Add to B1
    else:
        evict_page, _ = T2.popitem(last=False)
        B2[evict_page] = True  # Add to B2
```

شکل ۲۰: کد مربوط به بخش تاخیر در پاکسازی

۸ نتایج اجرای سیاست‌های مختلف روی حافظه‌ی نهان

Policy	Read Hits	Read Misses	Write Hits	Write Misses	Read Requests	Write Requests	Total Requests	Cache Hits	Cache Misses
LRU	۱۸۵۶۲۲۷	۱۲۴۲۵۰۸	۱۰۶۷۸۴۸	۹۲۱۳۴۹	۳۰۹۸۷۳۵	۱۹۸۹۱۹۷	۵۰۸۷۹۳۲	۲۹۲۴۰۷۵	۲۱۶۳۸۵۷
NHIT	۲۴۹۶۹۵۲	۶۰۱۷۸۳	۱۰۵۱۶۱۱	۹۳۷۵۸۶	۳۰۹۸۷۳۵	۱۹۸۹۱۹۷	۵۰۸۷۹۳۲	۳۵۴۸۵۶۳	۱۵۳۹۳۶۹
NHIT-LRU	۱۹۰۳۵۷۷	۱۱۹۵۱۵۸	۱۰۶۳۳۷۷	۹۲۵۸۲۰	۳۰۹۸۷۳۵	۱۹۸۹۱۹۷	۵۰۸۷۹۳۲	۲۹۶۶۹۵۴	۲۱۲۰۹۷۸
ARC	۲۱۱۷۱۷۸	۹۸۱۵۵۷	۱۲۱۳۲۶۳	۷۷۵۹۳۴	۳۰۹۸۷۳۵	۱۹۸۹۱۹۷	۵۰۸۷۹۳۲	۳۳۳۰۴۴۱	۱۷۵۷۴۹۱
LARC	۱۸۸۹۳۳۳	۱۲۰۹۴۰۲	۱۰۵۹۰۰۸	۹۳۰۱۸۹	۳۰۹۸۷۳۵	۱۹۸۹۱۹۷	۵۰۸۷۹۳۲	۲۹۴۸۳۴۱	۲۱۳۹۵۹۱
Belady	۲۷۷۵۶۴۰	۳۲۳۰۹۵	۱۶۵۶۱۱۵	۳۳۳۰۸۲	۳۰۹۸۷۳۵	۱۹۸۹۱۹۷	۵۰۸۷۹۳۲	۴۴۳۱۷۵۵	۶۵۶۱۷۷

جدول ۱: نتایج عملکرد حافظه‌ی نهان برای csv۴۲A. (اندازه‌ی حافظه‌ی نهان: ۸۴۸۰۷)

بدیهی است که در بین تمامی الگوریتم‌ها، الگوریتم Belady بهترین عملکرد را دارد زیرا الگوریتم بهینه است. در بین الگوریتم‌های باقی‌مانده، به ترتیب NHIT، ARC، NHIT-LRU، LARC و LRU بهترین عملکرد را دارند.

Policy	Read Hits	Read Misses	Write Hits	Write Misses	Read Requests	Write Requests	Total Requests	Cache Hits	Cache Misses
LRU	۱۵۳۵۴۴۵	۹۰۹۴۷۲۸	۳۵۲۱۳۹۱	۶۰۵۳۷۵۴	۱۰۶۳۰۱۷۳	۹۵۷۵۱۴۵	۲۰۲۰۵۳۱۸	۵۰۵۶۸۳۶	۱۵۱۴۸۴۸۲
NHIT	۲۹۹۵۶۷۷	۷۶۳۴۴۹۶	۳۶۵۹۷۳۲	۵۹۱۵۴۱۳	۱۰۶۳۰۱۷۳	۹۵۷۵۱۴۵	۲۰۲۰۵۳۱۸	۶۶۵۵۴۰۹	۱۳۵۴۹۹۰۹
NHIT-LRU	۱۵۱۴۱۱۳	۹۱۱۶۰۶۰	۳۲۵۵۹۲۷	۶۳۱۹۲۱۸	۱۰۶۳۰۱۷۳	۹۵۷۵۱۴۵	۲۰۲۰۵۳۱۸	۴۷۷۰۰۴۰	۱۵۴۳۵۲۷۸
ARC	۱۸۶۲۸۰۸	۸۷۶۷۳۶۵	۳۹۰۱۱۲۹	۵۶۷۴۰۱۶	۱۰۶۳۰۱۷۳	۹۵۷۵۱۴۵	۲۰۲۰۵۳۱۸	۵۷۶۳۹۳۷	۱۴۴۴۱۳۸۱
LARC	۲۳۵۹۵۶۵	۸۲۷۰۶۰۸	۴۵۱۶۱۲۸	۵۰۵۹۰۱۷	۱۰۶۳۰۱۷۳	۹۵۷۵۱۴۵	۲۰۲۰۵۳۱۸	۶۸۷۵۶۹۳	۱۳۳۲۹۶۲۵
Belady	۴۲۳۴۷۳۹	۶۳۹۵۴۳۴	۵۳۲۱۳۵۶	۴۲۵۳۷۸۹	۱۰۶۳۰۱۷۳	۹۵۷۵۱۴۵	۲۰۲۰۵۳۱۸	۹۵۵۶۰۹۵	۱۰۶۴۹۲۲۳

جدول ۲: نتایج عملکرد حافظه‌ی نهان برای csv۱۰۸A. (اندازه‌ی حافظه‌ی نهان: ۱۰۴۵۹۰)

در این فایل ردگیری، پس از الگوریتم Belady که بهینه است، به ترتیب الگوریتم‌های LARC، NHIT، ARC، NHIT-LRU و LRU بهترین عملکرد را دارند.

Policy	Read Hits	Read Misses	Write Hits	Write Misses	Read Requests	Write Requests	Total Requests	Cache Hits	Cache Misses
LRU	۲۰۳۴۲۰۵	۲۸۳۳۹۶۸	۹۰۷۶۵۷۰	۳۷۶۷۷۵۰	۴۸۶۸۱۷۳	۱۲۸۴۴۳۲۰	۱۷۷۱۲۴۹۳	۱۱۱۱۰۷۷۵	۶۶۰۱۷۱۸
NHIT	۱۸۵۴۰۷۰	۳۰۱۴۱۰۳	۸۵۶۷۱۴۹	۴۲۷۷۱۷۱	۴۸۶۸۱۷۳	۱۲۸۴۴۳۲۰	۱۷۷۱۲۴۹۳	۱۰۴۲۱۲۱۹	۷۲۹۱۲۷۴
NHIT-LRU	۲۰۶۵۴۶۵	۲۸۰۲۷۰۸	۸۸۶۹۰۶۴	۳۹۷۵۲۵۶	۴۸۶۸۱۷۳	۱۲۸۴۴۳۲۰	۱۷۷۱۲۴۹۳	۱۰۹۳۴۵۲۹	۶۷۷۷۹۶۴
ARC	۲۰۸۵۴۸۳	۲۷۸۲۶۹۰	۹۱۶۴۱۲۷	۳۶۸۰۱۹۳	۴۸۶۸۱۷۳	۱۲۸۴۴۳۲۰	۱۷۷۱۲۴۹۳	۱۱۲۴۹۶۱۰	۶۴۶۲۸۸۳
LARC	۲۰۳۰۶۴۱	۲۸۳۷۵۳۲	۹۳۳۴۶۷۳	۳۵۰۹۶۴۷	۴۸۶۸۱۷۳	۱۲۸۴۴۳۲۰	۱۷۷۱۲۴۹۳	۱۱۳۶۵۳۱۴	۶۳۴۷۱۷۹
Belady	۲۸۸۲۸۷۸	۱۹۸۵۲۹۵	۱۰۵۶۲۰۴۵	۲۲۸۲۲۷۵	۴۸۶۸۱۷۳	۱۲۸۴۴۳۲۰	۱۷۷۱۲۴۹۳	۱۳۴۴۴۹۲۳	۴۲۶۷۵۷۰

جدول ۳: نتایج عملکرد حافظه‌ی نهان برای csv۱۲۹A. (اندازه‌ی حافظه‌ی نهان: ۲۶۲۱۴۱)

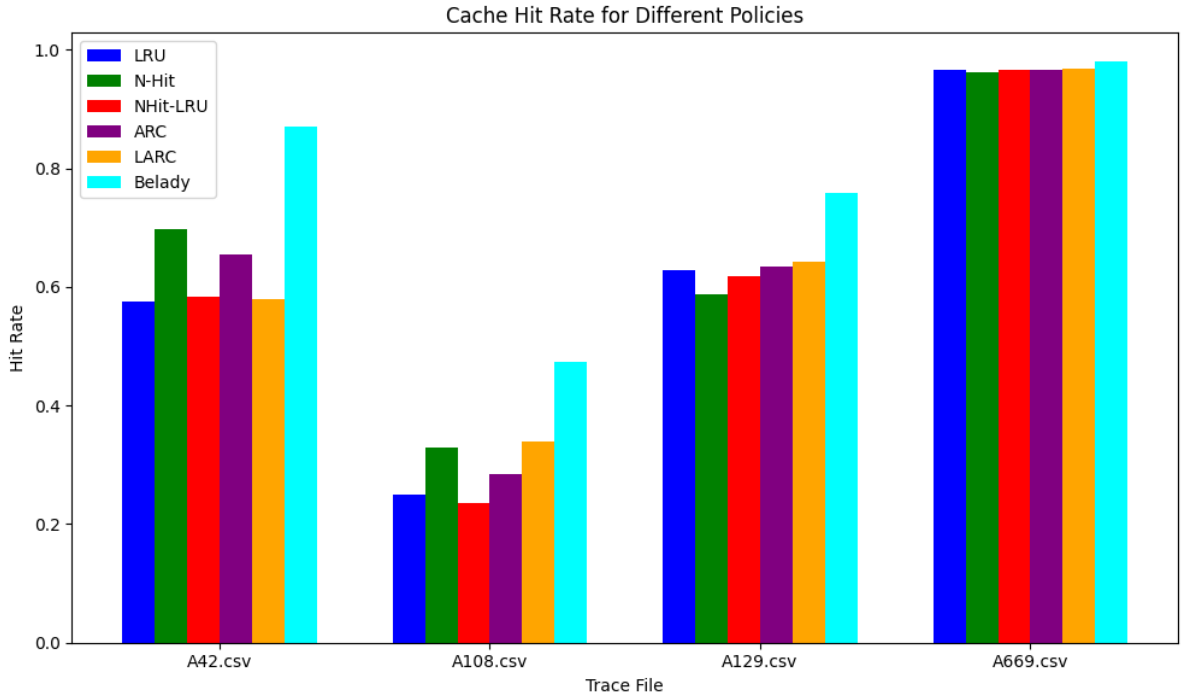
در این فایل ردگیری پس از الگوریتم Belady که الگوریتم بهینه است، به ترتیب الگوریتم‌های LARC، ARC، NRU و N-HIT بهترین عملکرد را دارند.

Policy	Read Hits	Read Misses	Write Hits	Write Misses	Read Requests	Write Requests	Total Requests	Cache Hits	Cache Misses
LRU	۲۵۳۸۳۵۱۷	۵۶۸۴۳۹	۹۶۳۴۷۸	۳۲۹۳۹۹	۲۵۹۵۱۹۵۶	۱۲۹۲۸۷۷	۲۷۲۴۴۸۳۳	۲۶۳۴۶۹۹۵	۸۹۷۸۳۸
NHIT	۲۵۳۳۳۹۴۱	۶۱۸۰۱۵	۸۶۳۷۹۴	۴۲۹۰۸۳	۲۵۹۵۱۹۵۶	۱۲۹۲۸۷۷	۲۷۲۴۴۸۳۳	۲۶۱۹۷۷۳۵	۱۰۴۷۰۹۸
NHIT-LRU	۲۵۳۵۵۰۱۷	۵۹۶۹۳۹	۹۵۸۲۶۰	۳۳۴۶۱۷	۲۵۹۵۱۹۵۶	۱۲۹۲۸۷۷	۲۷۲۴۴۸۳۳	۲۶۳۱۳۲۷۷	۹۳۱۵۵۶
ARC	۲۵۳۸۱۲۵۳	۵۷۰۷۰۳	۹۴۸۷۷۲	۳۴۴۱۰۵	۲۵۹۵۱۹۵۶	۱۲۹۲۸۷۷	۲۷۲۴۴۸۳۳	۲۶۳۳۰۰۲۵	۹۱۴۸۰۸
LARC	۲۵۳۸۳۴۳۳	۵۶۸۵۲۳	۱۰۱۸۳۴۸	۲۷۴۵۲۹	۲۵۹۵۱۹۵۶	۱۲۹۲۸۷۷	۲۷۲۴۴۸۳۳	۲۶۴۰۱۷۸۱	۸۴۳۰۵۲
Belady	۲۵۴۹۵۲۸۰	۴۵۶۶۷۶	۱۱۹۱۳۰۷	۱۰۱۵۷۰	۲۵۹۵۱۹۵۶	۱۲۹۲۸۷۷	۲۷۲۴۴۸۳۳	۲۶۶۸۶۵۸۷	۵۵۸۲۴۶

جدول ۴: نتایج عملکرد حافظه‌ی نهان برای csv۶۶۹A. (اندازه‌ی حافظه‌ی نهان: ۵۳۶۰۱)

در این فایل نیز پس از الگوریتم Belady که سیاست بهینه است، به ترتیب سیاست‌های LARC، LRU، ARC، NHIT-LRU و NHIT بهترین عملکرد را داشتند.

۹ تحلیل نرخ برخورد



شکل ۲۱: نمودار نرخ برخورد برحسب اسم فایل

در شکل ۲۱ مقایسه‌ای بین الگوریتم‌های جایگزینی مختلف در هر فایل و یک مقایسه کلی در تمامی فایل‌ها صورت گرفته است. در تمامی فایل‌ها، الگوریتم Belady بهترین عملکرد را دارد که قابل انتظار است زیرا الگوریتم بهینه است. همان طور که در شکل دیده می‌شود، در فایل ردگیری چهارم تمامی الگوریتم‌ها به الگوریتم بهینه نزدیک شده‌اند که می‌تواند نشان‌دهنده محلیت بالا در داده‌های این فایل شود. در واقع انتظار داریم با افزایش محلیت داده‌ها، عملکرد الگوریتم‌های جایگزینی به الگوریتم بهینه نزدیک شود. الگوریتم LRU قابل اطمینان است اما ممکن است بر اساس بار کاری همیشه بهترین سیاست نباشد. در هر چهار فایل ردگیری، الگوریتم ARC عملکرد بهتری نسبت به الگوریتم LRU دارد یا عملکرد بسیار نزدیکی به آن دارد که این نشان‌دهنده بهبود عملکرد الگوریتم جایگزینی در صورت استفاده از الگوریتم پویا و ترکیب دو الگوریتم LRU و LFU است. الگوریتم ARC عملکرد قابل قبولی در هر چهار فایل ردگیری دارد که نشان‌دهنده قابلیت تطبیق آن با بارهای کاری مختلف است. اما الگوریتم‌های N-Hit و LARC عملکرد متغیری دارند که نمایانگر این موضوع است که آن‌ها برای بارهای کاری به‌خصوصی به صورت قابل قبول عمل می‌کنند.